VIT BHOPAL UNIVERSITY

CSA-2001: FUNDAMENTALS OF AI & ML

Project Report

Scholarship Eligibility Predictor

ATHARVA DESAI

Registration Number: 25BCE10123

Branch: B.Tech — Computer Science and Engineering **Year:** First Year | **Course:** Fundamentals of AI and ML

1. The Problem I Chose

Scholarship selection in most colleges is done manually. Someone goes through each application and checks whether the student meets the criteria — CGPA, family income, number of achievements. This process is slow, sometimes inconsistent, and takes a lot of time when there are hundreds of applicants.

I thought this was a problem that machine learning could genuinely solve. If a model can learn from past scholarship data — which students were eligible and which were not — it can predict eligibility for new applicants automatically and consistently.

This problem is real, directly relevant to college students, and something I personally relate to. It also maps cleanly to what we learned in this course — classification using logistic regression.

2. Why This Problem Matters

- Scholarship selection is often slow and manual — automation speeds it up significantly
- Manual processes can be inconsistent — the same student might get different results from different evaluators
- An ML model applies the same criteria to every student fairly and consistently
- Students from low income backgrounds benefit most from faster and more accurate selection

3. My Approach

Algorithm	Logistic Regression — built from scratch using numpy
Dataset	200 students — 100 eligible, 100 not eligible
Features	CGPA, Family Income, Number of Achievements
Library	numpy only — no sklearn, no pandas
Split	80% training (160 students), 20% testing (40 students)

Step 1 — Dataset Creation

I created a dataset of 200 students using numpy. Eligible students have higher CGPA (mean 8.5), lower income (mean Rs 2 lakhs), and more achievements (mean 3). Not eligible students have lower CGPA (mean 6.5), higher income (mean Rs 6 lakhs), and fewer achievements (mean 1). I used np.clip to keep all values within realistic ranges.

Step 2 — Normalization

CGPA is between 6 and 10 but income is in lakhs. Without normalization the model pays too much attention to income just because the numbers are larger. I subtracted the mean and divided by standard deviation to bring all features to the same scale.

Step 3 — Train Test Split

I manually split the data — 80% for training and 20% for testing. I shuffled the indices first so the split is random. This is important so the model does not just memorize the order of the data.

Step 4 — Logistic Regression from Scratch

I built the entire model using numpy — no sklearn. The sigmoid function converts any number to a probability between 0 and 1. Gradient descent updates the weights and bias over 1000 steps to minimize the prediction error. No library function was used for training or prediction.

Step 5 — Evaluation

I evaluated the model using accuracy, precision, recall, F1 score, and a confusion matrix — all calculated manually using numpy array comparisons. This gives a much more complete picture of how the model performs than accuracy alone.

4. Key Decisions I Made

Why Logistic Regression?

Logistic regression is the most straightforward classification algorithm and is easy to implement from scratch using numpy. The sigmoid function directly gives a probability — which is useful here because it tells us not just whether a student is eligible but how confident the model is.

Why Build From Scratch Instead of Using sklearn?

Writing the sigmoid function and gradient descent loop manually forced me to understand what is actually happening inside the model. I understand what each line does — the weights, the bias, how they get updated, why the clip is needed. Calling sklearn.linear_model.LogisticRegression() would give the same result but I would not understand why.

Why Normalize?

Without normalization, income values in the hundreds of thousands would dominate the model completely and CGPA (which only goes to 10) would barely matter. Normalization makes sure all three features start with equal influence and the model learns which ones actually predict eligibility.

5. Challenges I Faced

The first challenge was understanding gradient descent. I knew the formula but did not immediately understand why we divide by len(y) when updating the weights. After thinking about it I realized it is to get the average gradient across all training samples — without it the update would be too large and the model would overshoot.

The second challenge was remembering to normalize the new student's data using the same mean and std from training — not recalculating it fresh. If I had recalculated, the new student's data would be on a different scale than what the model was trained on and the prediction would be wrong.

6. What I Learned

- How logistic regression works at the math level — sigmoid, weights, bias, gradient descent
- Why normalization is necessary before training any ML model
- How to implement a full training loop from scratch using only numpy
- What precision, recall, F1 score, and confusion matrix mean and how to calculate them manually
- That the same mean and std must be used for new data as was used for training data
- That building something from scratch teaches far more than calling a library function

7. What I Would Add With More Time

- Add user input so any student can type their own details and get a prediction
- Add matplotlib charts to visualize the data distribution and model performance
- Use a real dataset of actual scholarship applications
- Try a decision tree to compare which algorithm performs better on this problem
- Add a learning curve graph showing how accuracy improves over training epochs

8. Conclusion

This project taught me that machine learning is not about knowing which library function to call. It is about understanding the problem, preparing the data correctly, choosing the right algorithm, and evaluating honestly. Writing logistic regression from scratch using only numpy made me understand every step of what the model is actually doing.

The scholarship problem is also genuinely useful. A consistent, automated eligibility check is faster and fairer than a manual one. This is a real application of what we learned in this course and something that could be used in an actual college setting.

References

- numpy documentation — numpy.org
- Logistic Regression — Wikipedia — en.wikipedia.org/wiki/Logistic_regression

- Gradient Descent — 3Blue1Brown — youtube.com/@3blue1brown
- VIT Bhopal Scholarship Criteria — vitbhopal.ac.in